HONOURS MEET

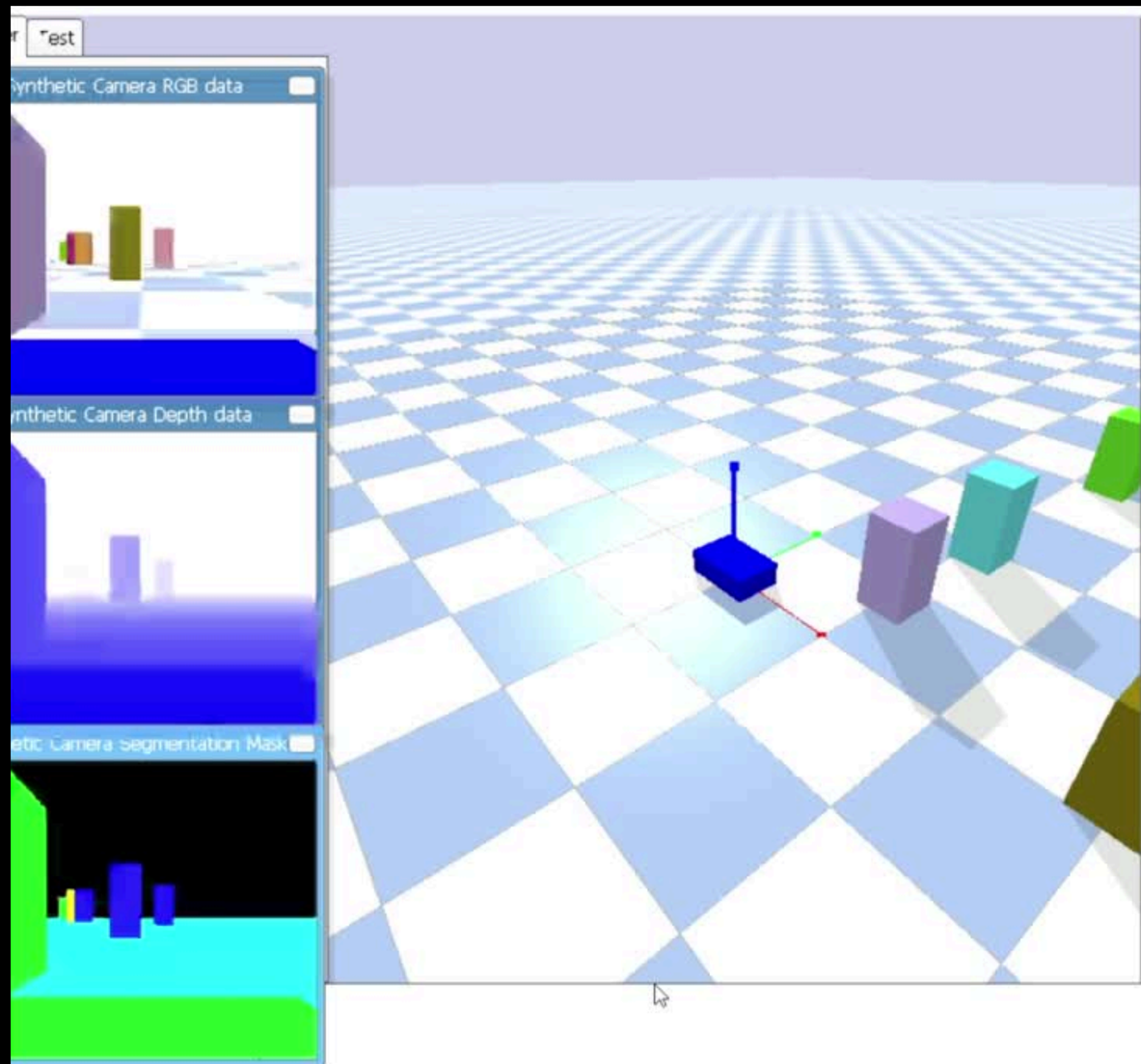
BY KEERTHI SEELA AND SNIGDHA STP

Project Objectives

1. Fixing the Tangential Motion of the Robot
2. Including the Barrier Lyapunov Function
3. Handling different motion scenarios
4. Using a Neural Network Architecture for a control action output
5. Exploring Reinforcement Learning framework- considering policy-based learning (control action as policy output)
6. Exploring Imitation Learning Framework

No Tangential motion

- Uses **ground truth** and Kalman Filter for robust state estimation
- Applies Lyapunov based control to ensure stable convergence to the goal
- Combines goal attraction and obstacle avoidance forces
- Enforces non-holonomic (car-like) constraints
- Eliminates **sideways motion** and moves only via forward velocity + steering



Different Motion Scenarios

1. Goal Seeking Objective

- In the absence of obstacles, the robot follows a Lyapunov-based attractive control law to move toward the target. The controller computes the position error e=xgoal−xe = x_{goal} - xe=xgoal−x .
- Kp and Kd values are dynamically generated by the neural network.
- The flow is: camera → feature extraction → NN → gains → Lyapunov controller → velocity command.
- This results in smooth, stable motion directly toward the goal while minimizing oscillations.

2. Obstacle-Aware Smooth Avoidance

- When obstacles are nearby, the system augments the attractive control with a barrier-based repulsive and tangential component. The barrier function h(x) = d-R ensures safety, where d is the distance to the obstacle and R is the radius of the robot.
- The flow is: detect nearby obstacles → compute barrier force → add tangential motion → combine with goal attraction.

3. Gain-Modulated Safe Navigation

- As the robot enters the caution zone, the controller modulates gains smoothly using a proximity factor.
- The flow is: measure distance → compute proximity factor → blend gains → apply control.
- This reduces speed and increases damping, making motion safer and more stable near obstacles.

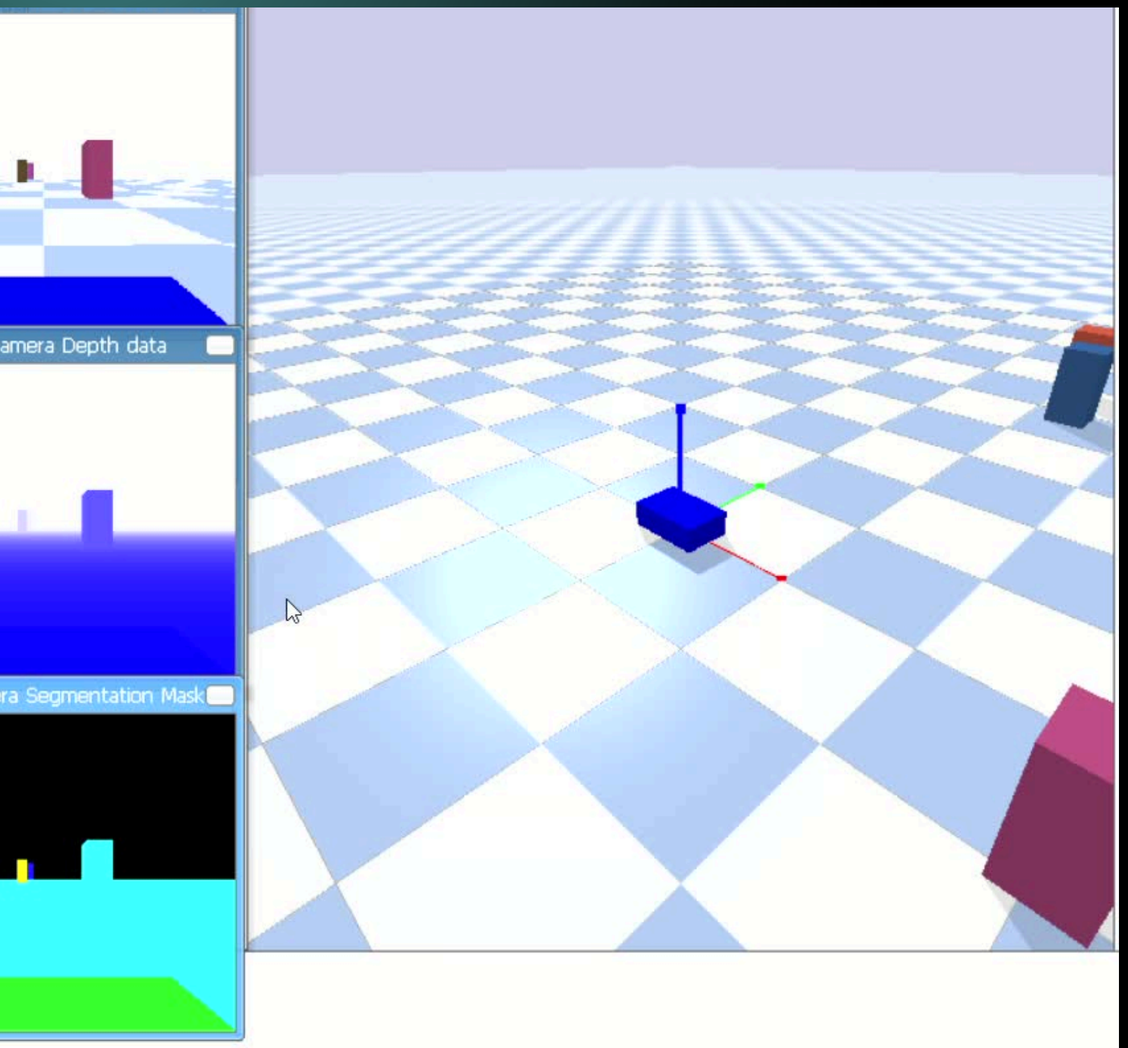
4. Front Blocked Reactive steering

- When an obstacle is detected directly ahead (within a cone), the controller reduces goal attraction and biases motion sideways. The attractive term is scaled down.
- The flow is: scan front arc → detect blockage → reduce forward pull → steer sideways.
- This ensures the robot does not push into obstacles and instead finds an alternate path.

5. Escaping Obstacles

- If the Obstacle is very close in front, the robot executes a motion with steering. The velocity becomes negative but the angualr velocity still aligns with a safe direction.
- The flow is: detect close front obstacle → compute reverse scale → steer
- This helps escape tight spaces and prevents getting stuck in local minima.
- There is also a feature when the obstacle is dangerously close, the system overrides all control and forces zero velocity v=0, omega =0. The flow is: check minimum distance → if below threshold → stop robot immediately. This acts as a final safety layer to prevent collisions when all other strategies fail.

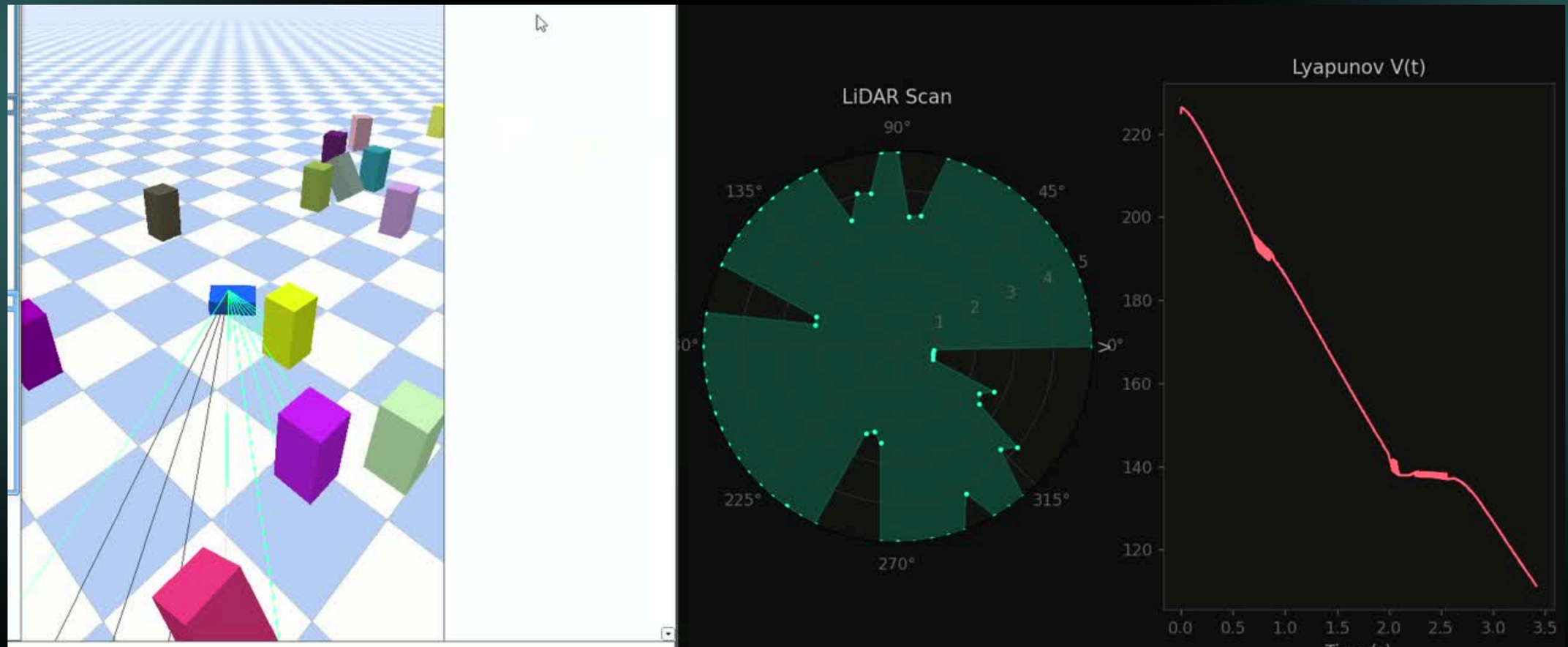
Zero-Knowledge Robot Navigation



- **No ground truth** used, robot estimates everything (position, velocity, heading) from sensors.
- Uses **Gyroscope (IMU)** for heading (orientation) and **Visual Odometry** for motion estimation
- Maintains uncertainty aware Kalman Filter and position error grows over time
- Builds a probabilistic target belief (updates as it re-observes target)
- Detects & tracks obstacles using **pure vision and tracking** (no prior map)
- Uses **Lyapunov and CBF control** for safe navigation and obstacle avoidance
- Operates in two phases: SEARCH (Before seeing target),NAVIGATION (After target seen)

LiDAR Based Autonomous Navigation

- Fully **perception-based** and no ground truth used in control (only sensors)
- Uses **IMU (gyro) and wheel encoder** for robot state estimation (EKF)
- Performs **LiDAR scanning** and detects obstacles via raycasting



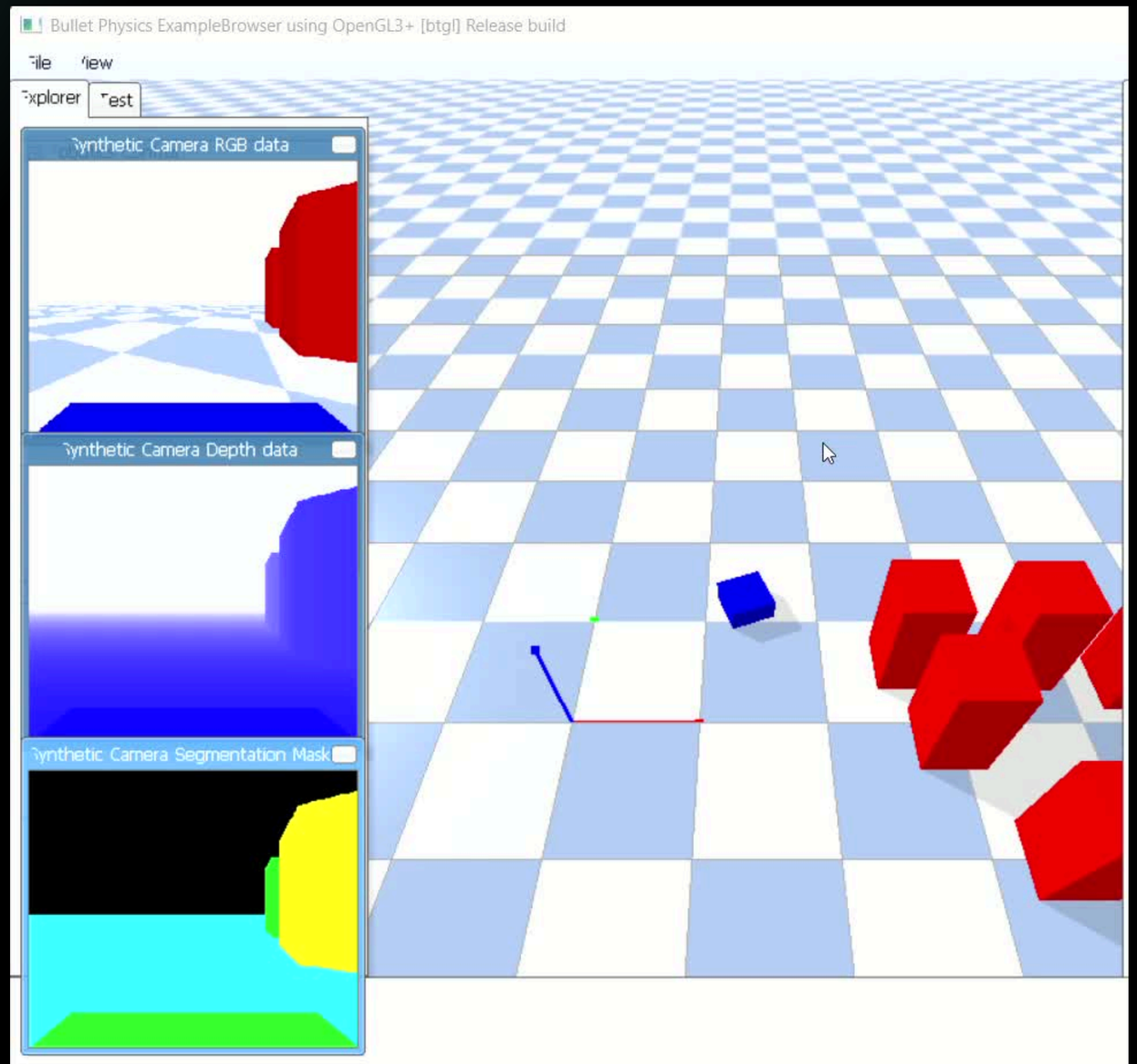
Implementing the RL Framework

1. The system begins by capturing an RGBA image of the robot's front view, which is passed into the feature extractor.
2. The extractor converts the image into grayscale and RGB formats and computes a compact 16-dimensional state vector consisting of goal-related features like
    a. distance
    b. angle,
    c. visibility of the green target)
    d. obstacle features (proximity and angles across five image columns plus overall density of dark regions)
    e. and motion features using optical flow (average pixel displacement between frames).

Implementing the RL Framework

3. This feature vector represents the robot’s perception of its environment and is fed into the train_gains module, where a neural network performs a forward pass to map these features to control gains Kp and Kd.
4. During training, the network updates its weights using policy gradient-based learning, optimizing a reward function that encourages reaching the goal efficiently while avoiding obstacles and maintaining smooth motion.
5. In the main control loop, the extracted features are continuously passed through the trained model to obtain Kp and Kd, which are then used in a PD controller to compute velocity commands, allowing the robot to steer toward the goal and adjust its motion dynamically based on the environment.

Implementing the scenarios with the RL framework

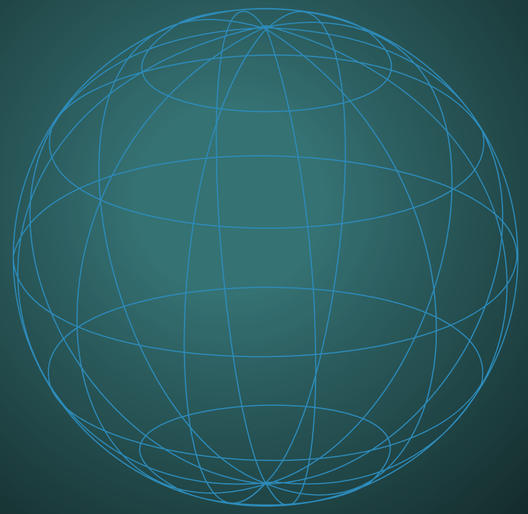


Autonomous Robot Navigation via Imitation Learning

- Task: Reach goal while avoiding static & dynamic obstacles
- Inputs:
  - Image (stacked frames)
  - Lidar (16 rays)
  - Goal direction + distance
- Output:
  - Linear velocity (v), Angular velocity (ω)

Approach:

- Use Lyapunov controller as expert
- Collect demonstrations (state → action)
- Train model using Behavioral Cloning (**Imitation learning**)



Multimodal Actor-Critic Network

- Image → CNN encoder
- Lidar + Goal → MLPs
- Fusion → shared representation
- Outputs:
  - Action (v, ω)
  - Value (critic)

Training:

- Dataset: expert demonstrations (image, lidar, goal → action)
- Loss: MSE (predicted vs expert action)
- Optimizations:
  - Adam + LR scheduler
  - Gradient clipping

Pipeline & Results

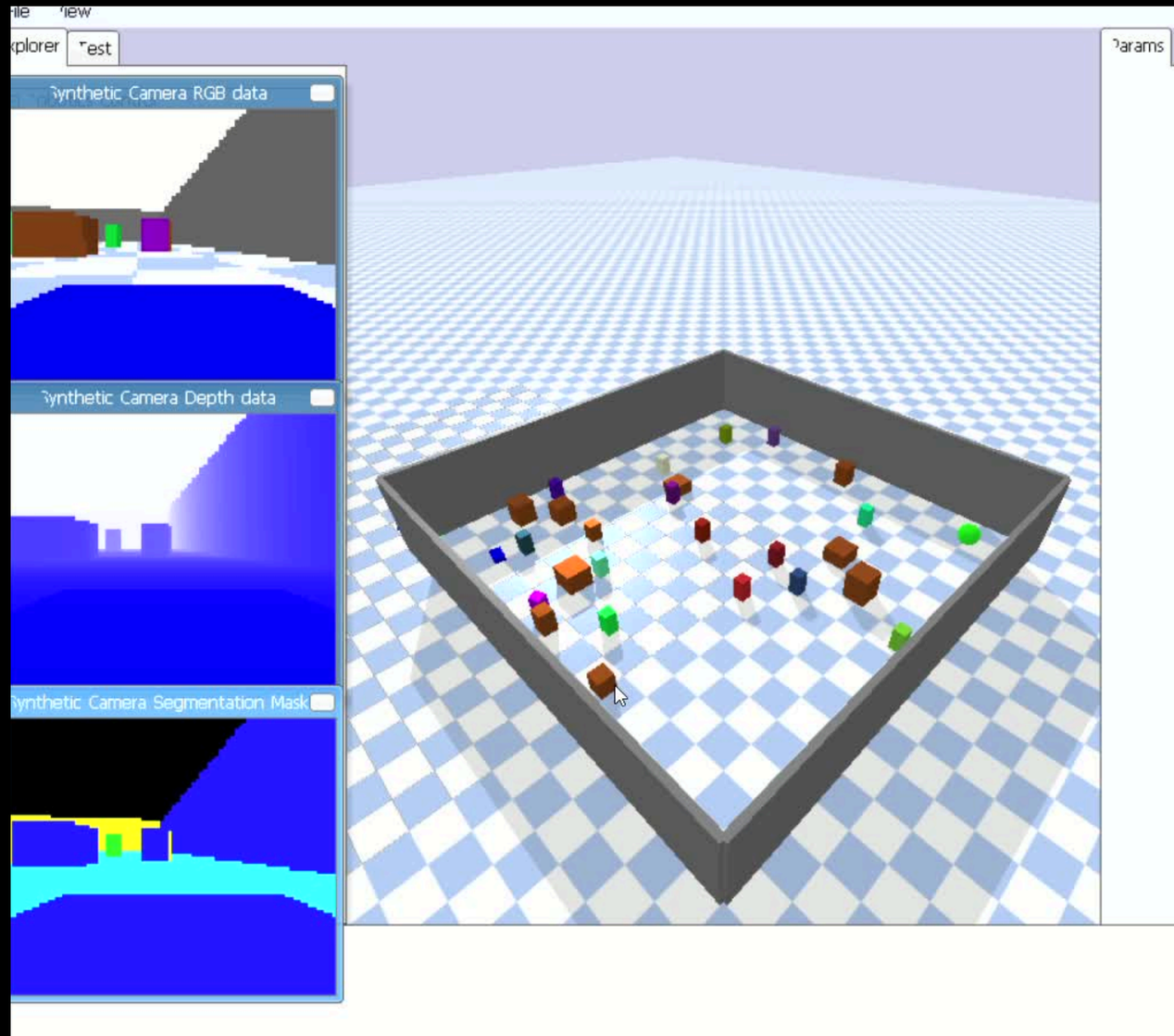
1. Simulated environment (PyBullet)
2. Expert generates safe trajectories
3. Dataset stored (demos.npz)
4. Train neural network
5. Evaluate learned policy

Outcomes:

- Learns navigation + obstacle avoidance
- Fast training (no exploration needed)
- Works well if environment ≈ training data

Limitation:

- Poor generalization to unseen scenarios



Thank You